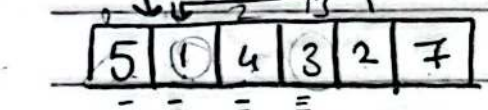
Veri yapıları ve algoritmalar

Search Algorithms
- → Linear
- → Binary Search
- → Jump Search
- → Interpolation Search
- → Exponential Search
- → hashing Search

Sorting Algorithms
- → Bubble Sort
- → Selection Sort
- → Quick Sort
- → Merge Sort
- → insertion Sort
- → heap Sort

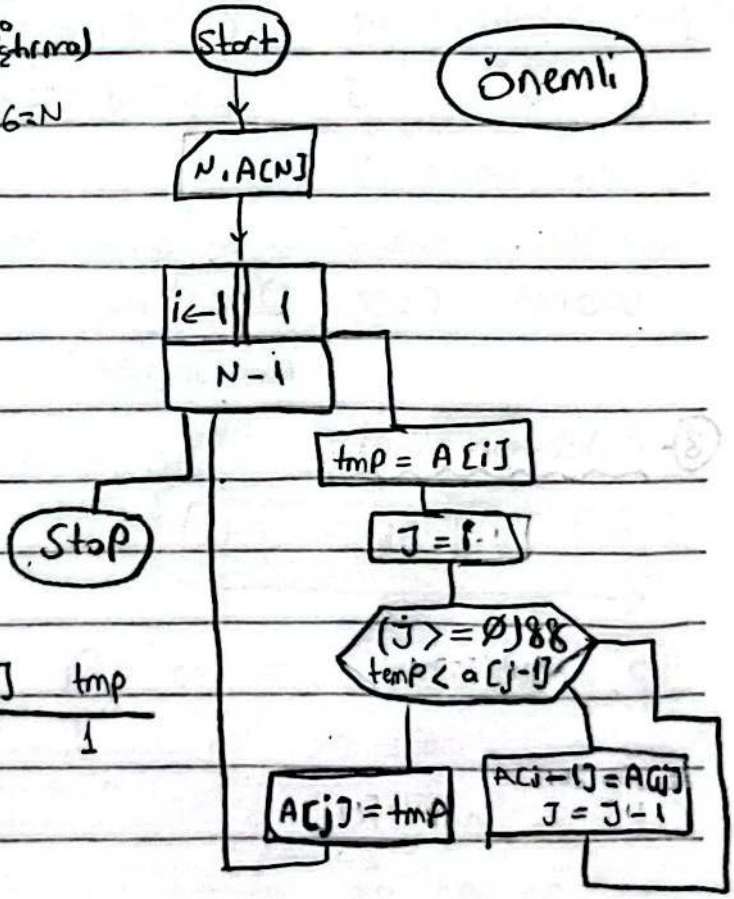
① insertion Sort: (araya sıkıştırma)



→ 1 5 4 3 2 7 ✓
→ 1 4 5 3 2 7
→ 1 3 4 5 2 7
→ 1 2 3 4 5 7 ✓✓

N = 6

i	A[i]	J	A[J]	tmp
→ 1	1	1	5	1



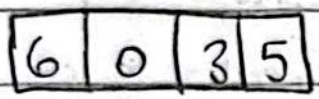
Worst case → $O(n^2)$ → $(n-1)$

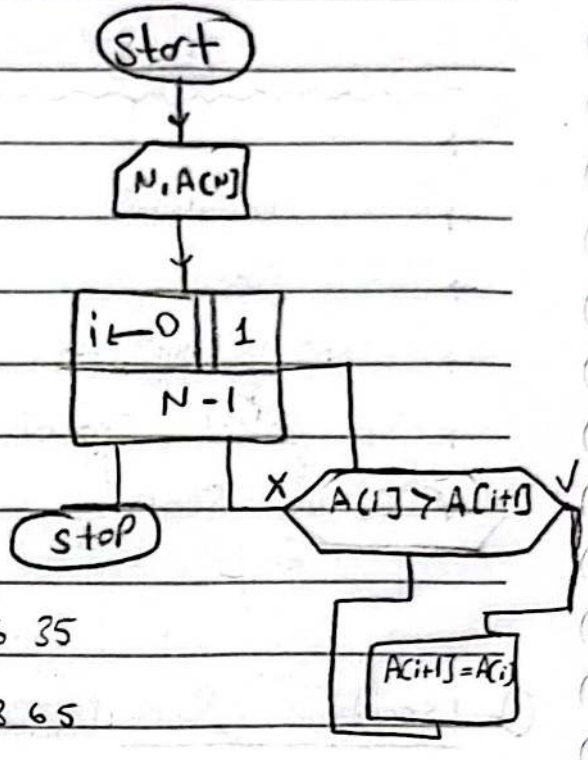
Best Case → $O(n)$ → $\left(\frac{n \cdot n-1}{2}\right)$

(!) Sıralıya yakın dizilerde kullanılır

Scanned with CamScanner

②- Bubble Sort





i=0 → 0 6 3 5

i=1 → 0 3 6 5

i=2 → 0 3 5 6

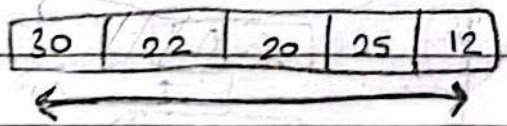
i	A[i]	A[i+1]	
0	6	0	6>0 ✓ 0 6 3 5
1	6	3	6>3 ✓ 0 3 6 5
2	6	5	6>5 ✓ 0 3 5 6
3	6	X	

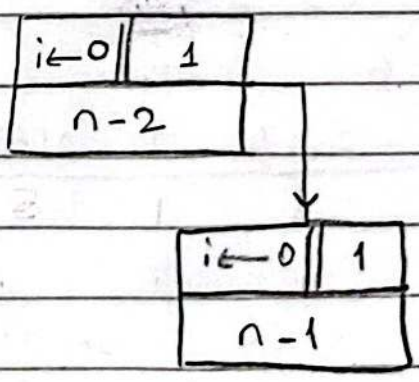
her elemanı sadece ondan sonra gelen elemanla karşılaştırıyoruz.

Worst Case: $O(n^2)$ ← hem kontrol hem değiştir

Best Case: $O(n)$ ← sadece kontrol

③- Selection Sort:





12 22 20 25 30

12 20 22 25 30

12 20 22 25 30

12 20 22 25 30 ✓✓

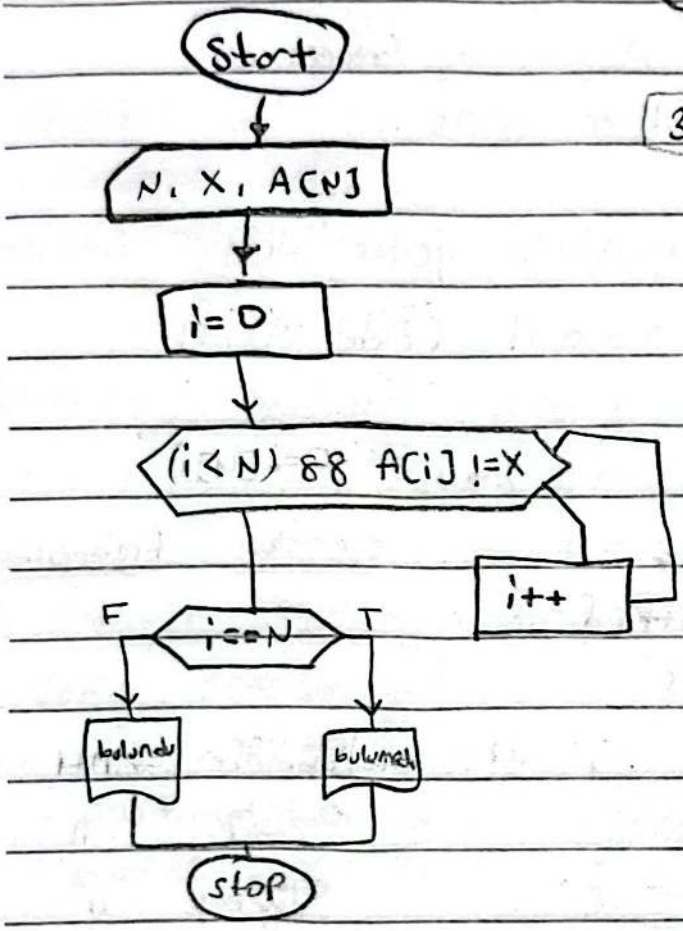
Best case = $O(n^2)$ Worst Case = $O(n^2)$

⊛ en küçük elemanı bul, başa yerleştir...

Sona Cinlayo

①- Linear Search

while $O(n)$



3	5	1	2	8

X=2
N=5

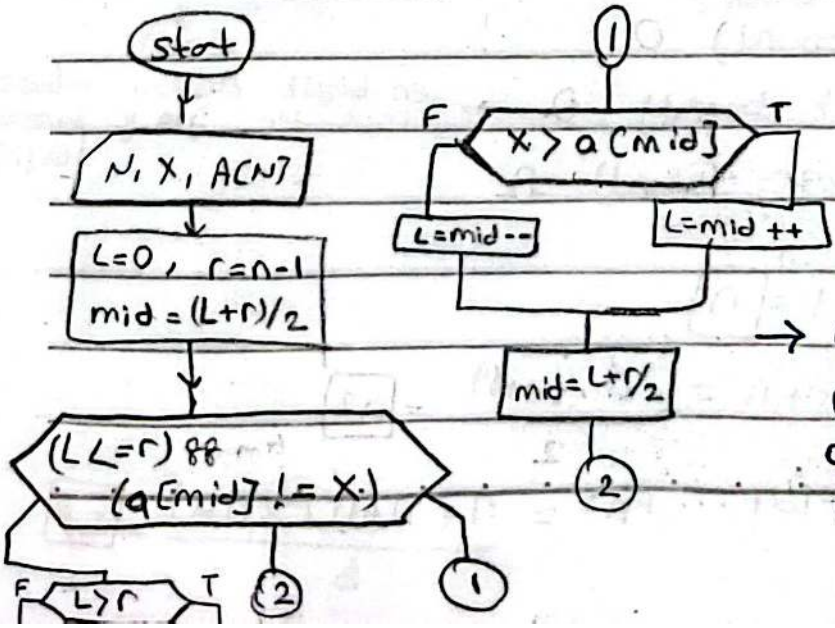
→ for kullanılmaz asla aaaaa:
- anlaşılırlık açısından çok kötü
- assembly dilinde işlemesi daha zor
- break gerektirir.

②- Binary Search (büyük veriler için)

çok sorulur ***

O(log n)

L		(x=9)		r
2	7	9	11	17



→ Sıralı dizide Decrease and conquer yöntemi kullanır

→ ortayı bulup aradığın sayıyla kıyaslıyoruz büyükse sola küçükse sağa

→ aradığımız sayı dizide olmasaydı L ve r eşit olur sonra olurdu o yüzden koşulumuz bu L>r

System dependent effects :

① - hardware : memory, cache ...

② - Software : compiler, programming language ...

③ - System : network, other apps ...

→ Çok büyük verili yapılarda yedek diziler kullanılmamalı

→ a++ (inc Ax) daha hızlı a = a + 1 (Add Ax, 1)

****

```
int countt = 0, i, n;
for( i=0 ; i<n ; i++) {
    if( a[i] == 0) {
        count ++ ;
    }
}
```

operation	frequency
variable detect	3
assignment	2
less than comp.	n+1
equal to comp.	n
array access	n
increment	(n - 2n)

$(6n+6) - (5n+6)$ arası $\leftarrow$ $O(n)$

Asymptotic notation

→ Big - O (upper bound) O

→ Big - theta (tight bound) Θ → en büyük elemanı bulmak için tüm diziyi taramak Θ(n)

→ Big - omega (lower bound) Ω

- $\sum 1 = 1+1+1+\dots 1 = n$
- $\sum i = 1+2+3+\dots+n = \frac{n\cdot(n+1)}{2} = n^2$
- $\sum i^2 = 1+4+9+16+\dots+n^2 = \frac{n(n+1)(2n+1)}{6} = n^3$

✳ Inplace Sorting : aynı dizi üzerinde yedek dizi oluşturmadan işlem yapmak.

- Bubble Sort
- Selection Sort
- Insertion Sort
- Quick Sort

✳ Merge Sorting : inplace sortingin tersi çünkü daha fazla yer gerektiriyor (for storing temp arrays)

✳ Stable Sorting : Birinci sırayı bozmadan ikinci veriyi sıralamasıdır.

- insertion Sort ✓✓
- bubble Sort ✓✓

selection → unstable ✗✗

2002	Zeynep →	2002	Zeynep
2003	mahmut	2003	Erce] sıralandı
2003	Erce	2003	mahmut
2004	berk	2004	berk
↓		sıralı ✓	sıralı ✓
sıralı			

Recursive Programming

① base case
② recursive case
③ ensure recursion terminates
④ Return the result

! recursive kullanmanın dezavantajı:

```
f(x);          void f(int x){
               f();   → gittiği
               }      ← dönecegi yeri saklar
                      ve
                      ek yavaşlatır
```

durma şartı doğru olmazsa → infinity loop

(*) O yüzden Sadece gerektiği zaman kullanılmalı

without recursive

```
int main(){
int result, x;
int factorial(int);
printf();
scanf();
result = factorial(x);
}
```

```
int factorial (int x){
int i, result=1;
for(i=1; i<=x; i++){
   result *= i;
}
return result;
}
```

with recursive

```
int factoril (int x){
if (x == 0){        → base case
   return (1);
}
else
   return ( x * factorial (x-1));
}
```

Recursive step

3 * fac(2) = 6
2 * fac(1) = 2
1 * fac(0)
1

→ Eeach recursive calls create a new copy of the function variables, arguments → in the stack on memory

→ Overhead of maintaining the stack
Recursion is slower than non-recursive

Kucuk not: fibonacci recursive'de yapsın hata:

fib(n) = fib(n-1) + fib(n-2)

fib(5) = fib(4) + fib(3)

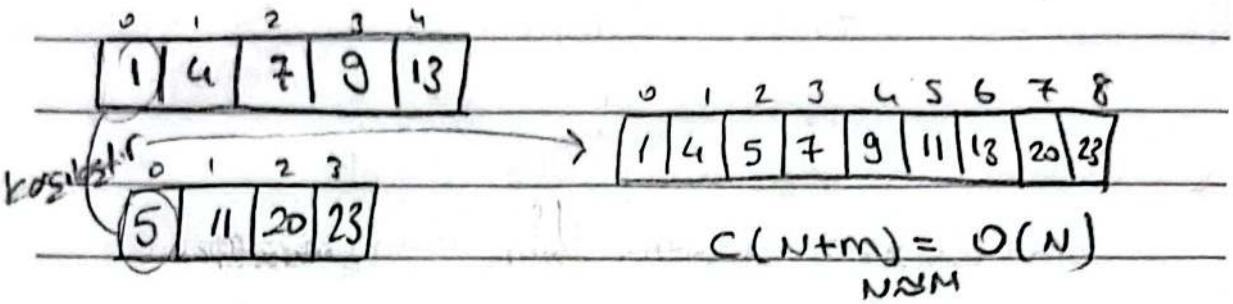
fib(3) + fib(2) — fib(2) + fib(1)

iki kere hesaplanıyor

Binary recursive (bir avantajı yok bar-bar döner)

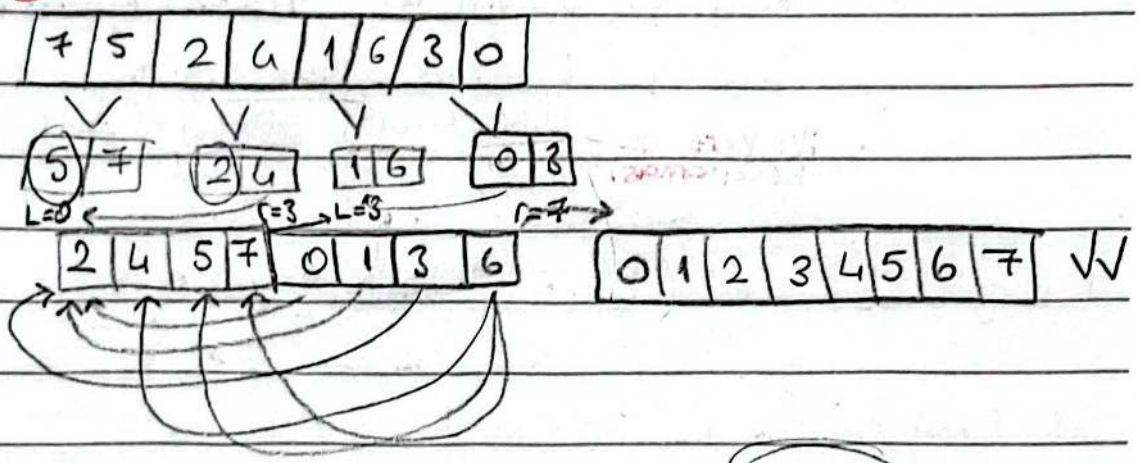
```
int Binary (int a[], int R, int L, in x){
  int mid;
  if ( L>r) {
    return (-1) ; }
  mid =(L+r)/2
  if ( a[mid] == x) {
    return mid;
  else if ( a[mid] < x)
    return binary( a, mid+1, r, x);
  else
    return binary( a, mid-1, r, x);
}
```

Merge : sıralı iki diziyi birleştirme (karşılaştırma yaparak)



$C(N+m) = O(N)$
$N \approx M$

Merge Sort



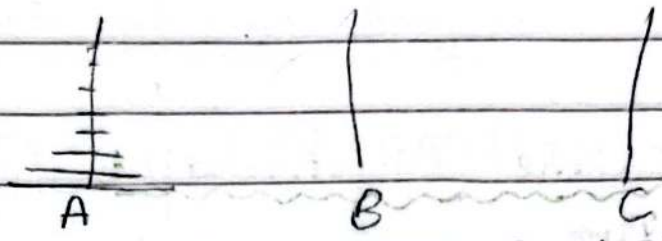
→ yedek diziyi kullanıyor ama $n \cdot \log n$ zamanında çözer

```
void mergeSort(int a[], int l, int r){
    if(l<r){
        m=(l+r)/2
        mergeSort(A,l,m);
        mergeSort(A,m+1,r);
        merge(A,l,m,r);
    }
}

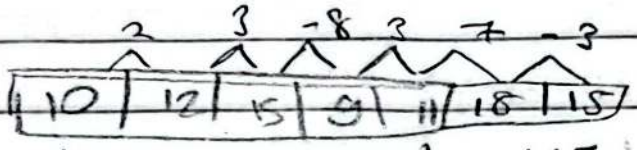
main()
mergeSort(a,0,n-1);
```

```
void merge(A,l,m,r)
int *B;
B=(int*)malloc(...)
i=l, k=0, j=m+1;
while((i<=m) && (j<r)
    if(A[i] <= A[j])
        B[k++]=A[i++]
    else
        B[k++]=A[j++]
while(i<=m)
    B[k++]=A[i++]
while(j<=r)
    B[k++]=A[j++];
for(ib=0;ia=l;ib<k;ib++;ia++)
    A[ia]=B[ib];
free(B)
```

Ödev

① 

diskleri C ye yerleştir
kuralı boyuşun üstüne gelebilir
aynı tersi olmaz

② 

← hisse fiyatları

artışın en yüksek olduğu parçayı
ne zaman alsın ne zaman satsın
↳ en yüksek karı elde ederim
bu işte iki döngü - global max
local max

(En kısa yol)

→ normal
→ recursive
→ internet

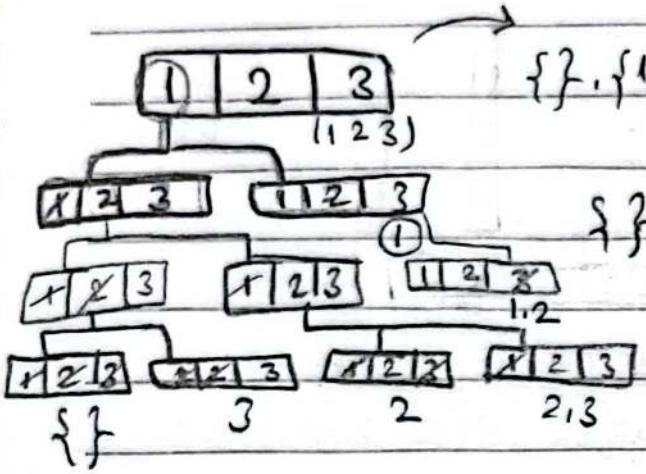
Lab -1-

① Sudoku Çözme Algoritması:

- Backtracking
- Stop koşulu: no available number veya tüm boşluklar doldusa...

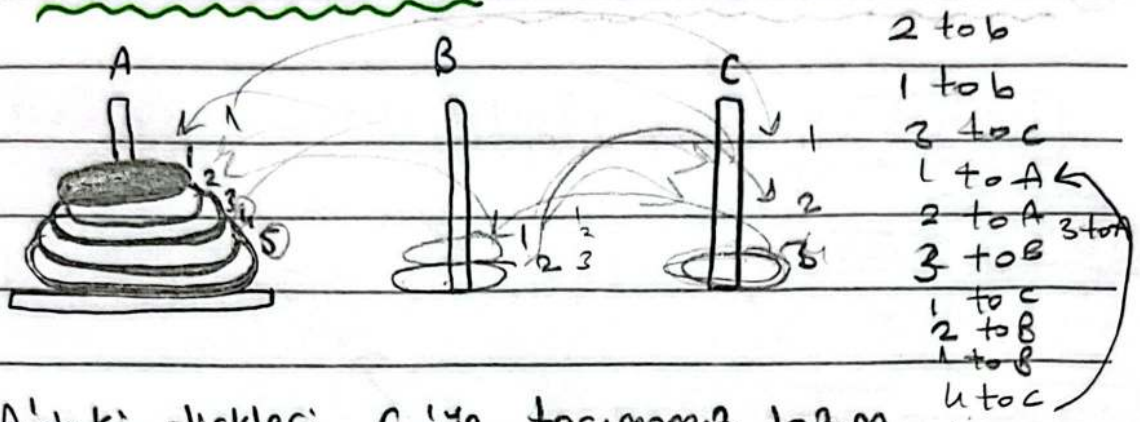
```
int solveSudoku(int arr){
  int currX, currY;
    if( findEmptyLocation (arr, &currX, &currY)==0)
      return 1;
    for(i=1; i<10; i++){
      if (checklocation(arr, currX, currY, num)==1){
        arr[currX, currY] = num;
        if( solve Sudoku (arr))
          return 1;
        arr[currX, currY] = 0;
      }
  }
return 0;
```

② All Subsets of an Array :




```
Void calcSubset(int * arr, int n, int * subSet, int indx)
{ int i ;
   PrintLastSubset (subset);
    for( i = index ; i<n ; i++) {
        appendToSubset (subSet , arr[i]);
        calcSubset(arr, n, subSet, i+1);
        removeLastSubset (subSet);
    }
}
```

③ Tower of Hanoi



A'daki diskleri C'ye taşımamız lazım

→ küçük diskin üstüne büyük disk konulmaz

```
Void PlayTOH (int n, char fromRod, char toRod, char auxRod)
{ if (n==0){
      return;}
  PlayTOH (n-1, fromRod, auxRod, toRod);
  Printf(" Disk %d moved form %c to %c ....
  PlayTOH(n-1, auxRod, toRod, fromRod);
return;}
```

n	F	T	Aux
3	A	C	B
-2	A	B	C
1	A	C	B
Recursive	∅ → 1 → A → C		
	∅ 2 → 2 → A → B		
1	C	B	A

6-

④ Bonus Length of longest palindromic Substring

input : S = "banana"

output : 5 --- 'anana'

→ start = end ? banana

→ start+1 = end ?

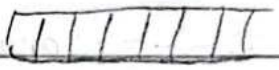
→ start = end-1 ?

esit sayıdaki harfler

```
int longestPal(char* str, int start, int end, int count){
    if( start > end)
        return count;
    if(start == end)
        return count+1;
    if ( str[start] == str[end]){
        count = longestPal(str, start+1, end-1, count+2);
return findMax(count, findMax(longestPal(str, start+1, end, 0)
, longestPal(str, start, end-1, 0)));
    }
    find ....
```

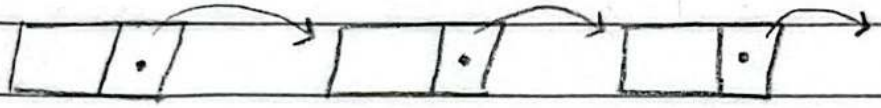
hafta -4-

Liste → Array

Static → int no[50];  50

Dynamic → int *no; no = (int *) malloc (n * sizeof(int));

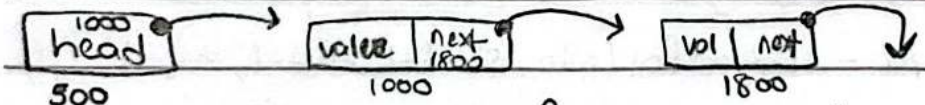
Linked Lists



- Sequential Access (direkt erişim yok)
- graflarda, ağaçlarda çok kullanılır, çok avantajı var.

① Single Linked List:

sadece tek yönde bağlanan yapıya denir



→ forword only!

- insert (add)
- delete
- list (search) print

```
include <stdlib.h>
typedef struct Node {
    int value;
    Struct Node *next;
} NODE;
NODE *head;
```

→ adress saklayacak kadar yer tutuyor (Pointer)

→ tüm fonksiyonlarda geçecegsinden eminiz. o yüzden global tanımladık

⊙ insert (adding):

→ front

→ tail (end)

→ anywhere you want (homework)

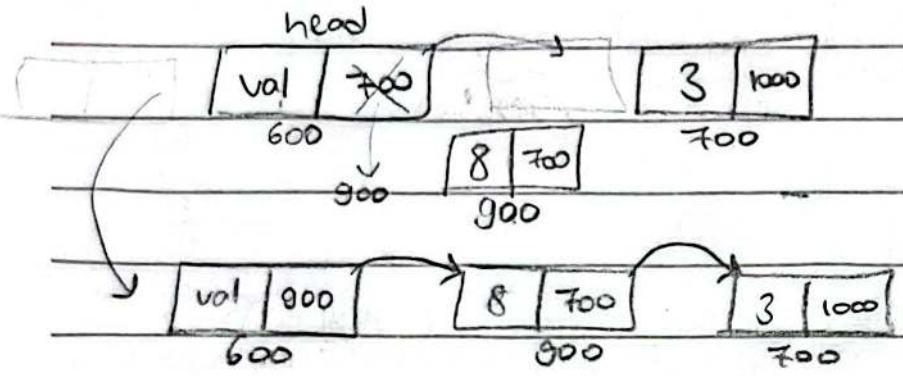
→ insert at front and at end:

```
Node *creatNode(){
    Node *p;
    p = (Node *) malloc (sizeof (Node));
    if (p == NULL) {
        printf("failed");
        exit(0);
    }
    p->next = NULL;
    return p;
}

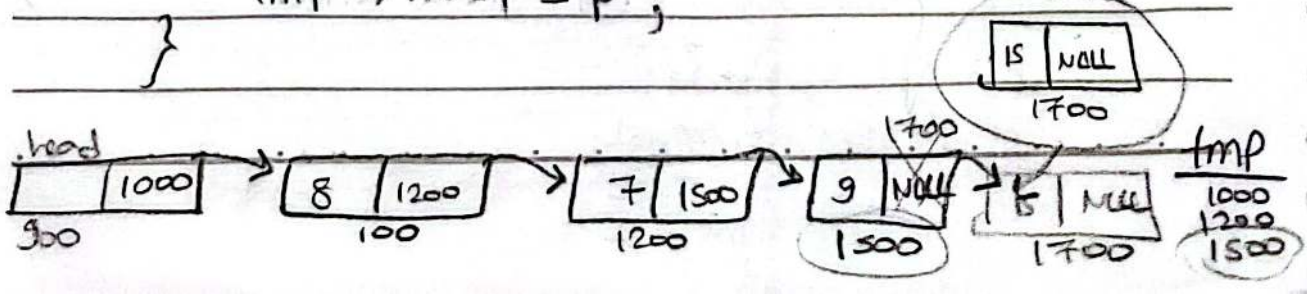
int main()
{ NODE *p;
    head = NULL;
    p = creatNode();
    insertatfront(p,8);
    P = CreateNode;
    insertAtEnd(P,5);
```

```
Void InsertAtFront (Node *P, int valee){
    P -> value = value;
    P -> next = head;   // head'in tuttuğu adresi P'ye ver
    head = P; // head P'yi tutsun
}
```

yeni head ekleme olmuyor.

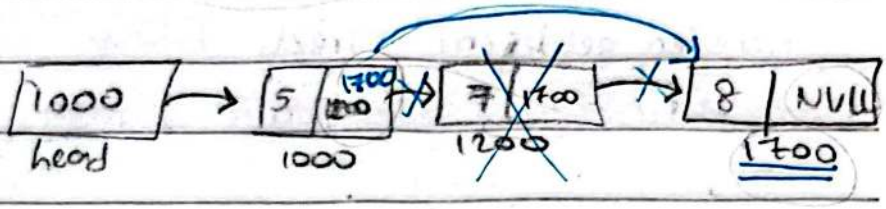



```
Void insertATEnd (Node *P, int value){
Node *tmp;
P -> value = value;
if (head == NULL) { // list boş
        head = P;
else
    tmp = head;
    while( tmp -> next != NULL) {
            tmp = tmp -> next; // head'in tuttuğu adres
    }
    tmp -> next = P;
}
```

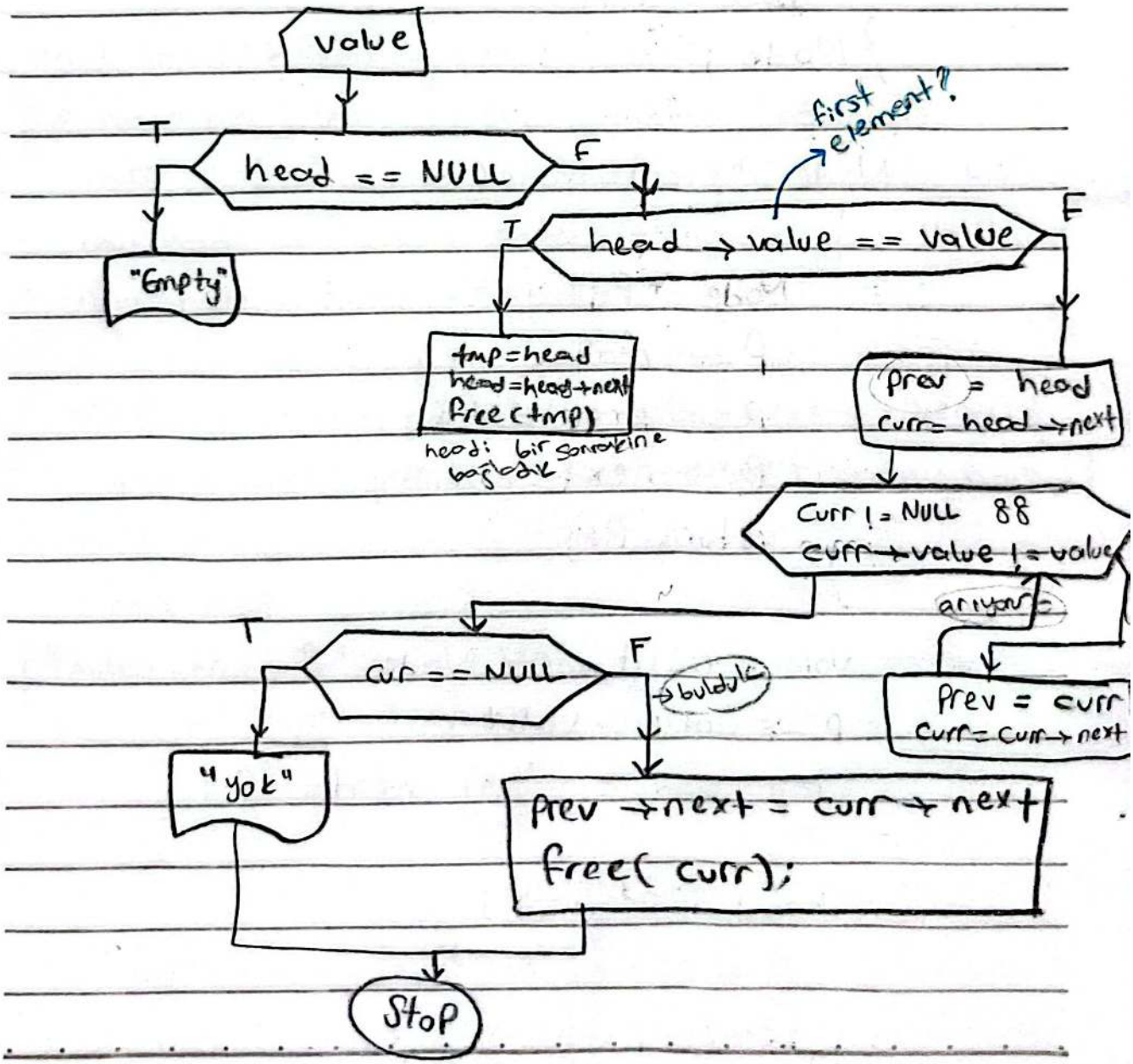


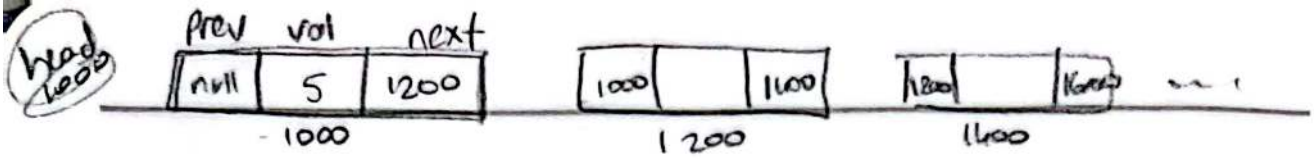
Delete :

→ Inside
→ First
→ end



void delete (int ValueToDelete)





Double Linked List :

nereden geldiğini sürekli tutarak gerektiğinde kullanmaktır.

```
typedef struct Node {
    int value;
    Node *prev;
    Node *next;
} Node;

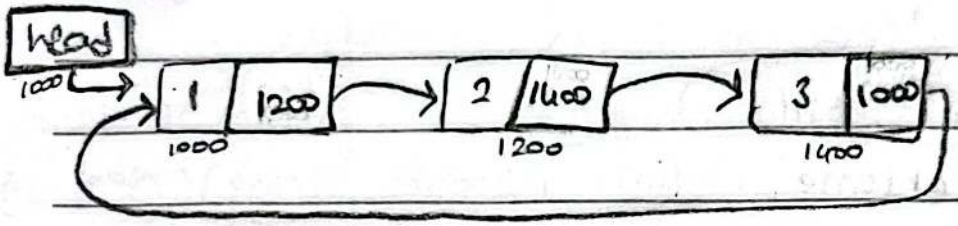
Node *createNode()
{
    Node *p;
    p = ... malloc ...
    p -> prev = Null;
    p -> next = Null;
    return p;
}

Void insertAtfront( Node *p, int value) {
p -> value = value;
if (head == Null) head = p;
else {
head -> prev = p;
p -> next = head;
head = p;
}
}
```

Scanned with CamScanner

③ Circular Linked List

Son node'un next'i null değil ilk node'un adresini tutar.




```
void insertAtBeginning (head, data){
// yer açma işlemi : malloc ... new_node
  node *last = *head; // son düğüm head'in tuttuğunu tuttu
  new_node -> data = data;
  new_node -> next = *head;
  if the list isn't empty traverse to last node
  if(*head =! NULL){ while(last -> next != *head){
    last = last -> next;} last -> next = new_node;}
  else { if the list empty, then make the new node -> head
    new_node -> next = new_node;}
  *head = new_node;}
```

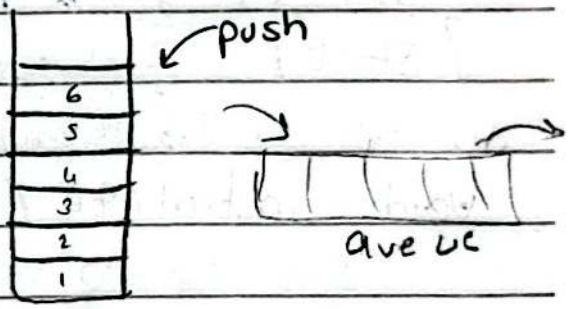
Linked List

- Random Access
- contigious memory
- slow on delete
- static memory

Array

- Sequential Access
- non-cont. memory
- fast in delete
- Dyromic memory

⊛ STACK (yığın)



first in last out FILO
Last in first out LIFO

→ Applications:

1. evaulation of arithmetic op. A + (B - C (D * E(G)))
2. Recursion func. usage
3. Backtracking (BFS) (control z) (web going back)
4. Paranthesis matching
5. conversion of Decimal to other num. systems.

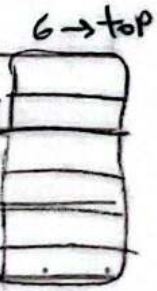
* top → en üstteki elemanın indisini tutar.
* Push → verilen değerle yeni node oluştur. en üste yerleştirilir
* Pop → en üstteki düğüm çıkarılır ve geri döndürülür

* Stack Overflow : bu hatayı almamın nedeni Stack sırasında durma koşulunu (recursive) doğru ayarlamamızdır.

```
#define MAX 1000
typedef stack {
   int items[MAX];
   int top ; // indis sayısı tutar
} Stack;

void initialize (Stack *S){
    S -> top = 0 ; }

int isEmpty (Stack *S){
    if( S -> top == 0) {
     printf(" Empty /n");
     return 1 ;
    }
   else { return 0 ; }
}
```




```
int isFull (Stack *S){
    if( S -> top == MAX){ return 1 ; }
    else { return 0 ; }
}
```

```
int push (int value, Stack *S) {
    if (isFull(S)) {
        return 0; }
    else {
        S -> items[S -> top] = value;
        (S -> top)++ ; return 1;
    }
}

int pop (stack *S, int value) {
    if (isEmpty(S)) { return 0 ; }
    else {
        (S -> top)-- ;
        * value = S -> items[S -> top];
        return 1;
    }
```

Asıl amaç kullanıcının sadece en üstte ekleme veya çıkarma yapabilmesine olarak sağlamak. Yani sadece son değişikliğe izin vermek.

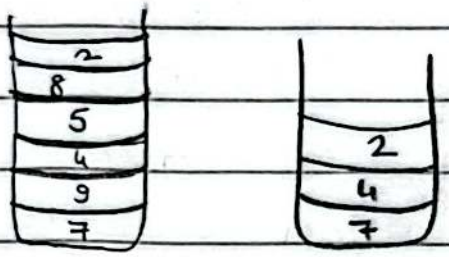
```
int main() {
  stack SS;
  initilize(SS);

  return 0;
}
```

Soru:

herhangi bir anda stack'teki en küçük sayıyı nasıl bulurız.

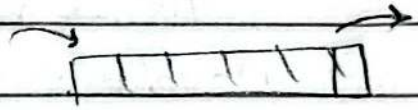
→ yeni bir stack tanımla, minimum değerler.



7 9 4 5 8 2
2 8 5 4 9 7

→ Ana stack'ten top alınırsa yedek stack top'uyla karşılaştır. eşitse onu da silesiz.

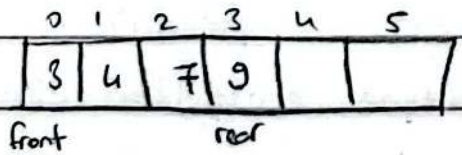
⊛ QUEUE : iki uçlu stack



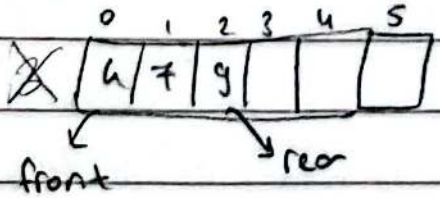
first in, first out.

enqueue : eleman ekleme

dequeue : eleman alınma



enq(3); enq(4); enq(7); enq(9); x = deq();





→ hepsini kaydırmamız gerekecek. böyle karmaşıklık çok

```
void initQueue(){
  int front = -1, rear = -1;
}
```

```
int isEmpty(){
  if(front == -1){
    return 1; }
  else { return 0; }
}
```

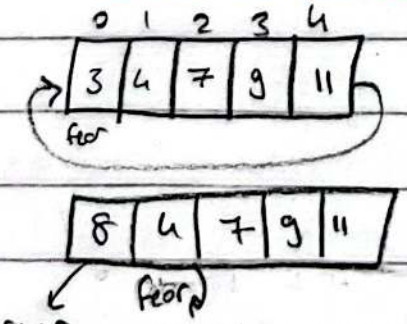
```
int isfull(){
  if(rear == MAX-1){
    return 1;
  else { return 0; }
}
```

```
typedef struct {
  int front
  int rear
  int items[MAX];
} QUEUE;
```

(*) int deQueue(){
  if(isEmpty())
    return 0;
  else
    *X = q[front++];
  return 1;
}

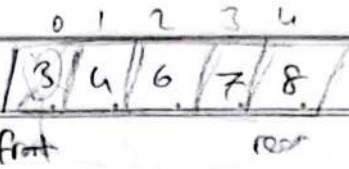
(+) int enQueue(int value){
  if(isfull())
    return 0;
  else
    rear++;
    q[rear] = value;
  return 1;}

(*) Circular Queue



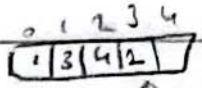
X = deq() // 3 silindi front = 4
enq(8) // 0. index'e 8 geldi

Alan doldu



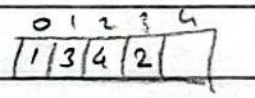
farklı olan kodlar:

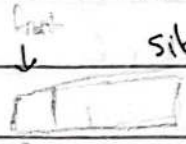
```
int Enqueue ( Queue * q , int value){
    if (isEmpty()) return 0;
    else{
        q → rear = (q → rear +1) % MAX ;
        q → item [q → rear] = value ;
        if( q → front == -1) {
            q → front = 1 ; }
        return 1;
    }
}
```



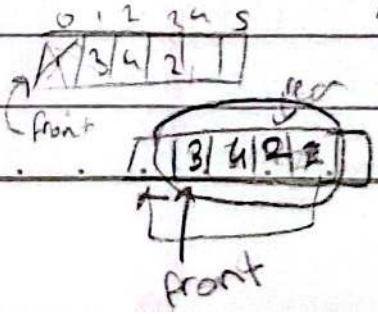
2=rear
3%4=3

```
int dequeue ( Queue * q) {
    if (isFull()) return 0;
    else {
        sildim// Value = queue → items[queue → front];
        = if ( queue → front == queue → rear){
            queue → front = -1;
            queue → rear = -1
        }else {
            queue → front = (queue → front + 1) % MAX
        }
    return val.
```



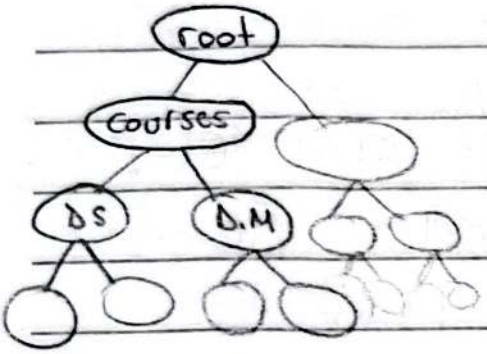


1%4 = 1

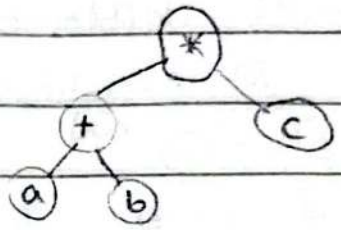


tree data Structure

- File System



- Syntax tree



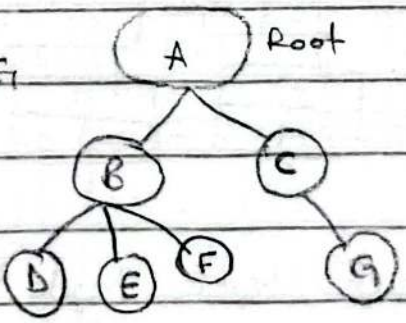
(A+B) * C

:

(*) Machine Learning
Decision Tree

→ Linear Data Sets:
Array, Linked List, stack ...
→ Non-Linear → tree

Tree terminology

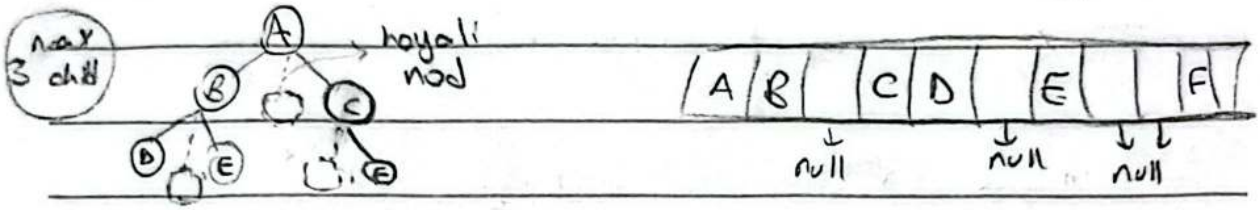


→ Nodes: A, B, C, D, E, D, E, F, G
→ Internal Nodes: A, B, C
(has at least one child)
→ Parent: B child: D, E, F
→ Edge: E → G tek yönlü bağlar (bazen değeri olur)
→ Degree: total number of children of a node. A: 2 B: 3
Leaves: D, E, F, G (farklı yüksekliklte olabilir)

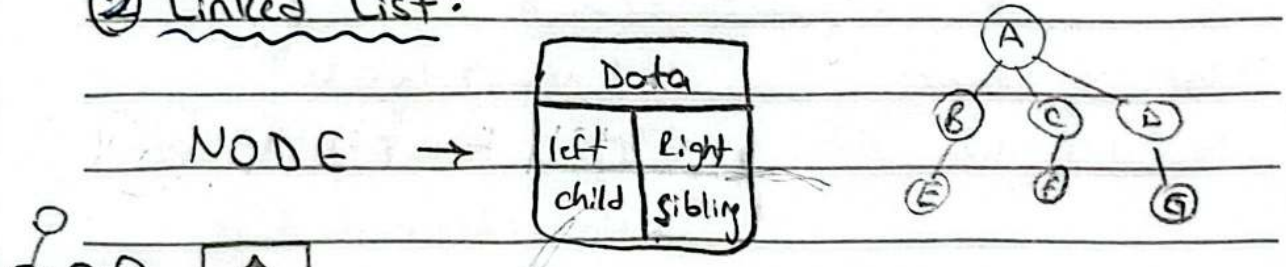
Tree Representation

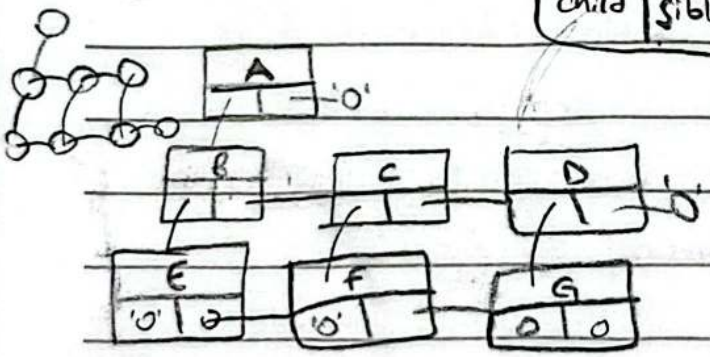
① Array:

ağaç yapısı için dizi kullanılacaksa dizide boşlukların bırakılması depolama açısından sorun yaratır



② Linked List:



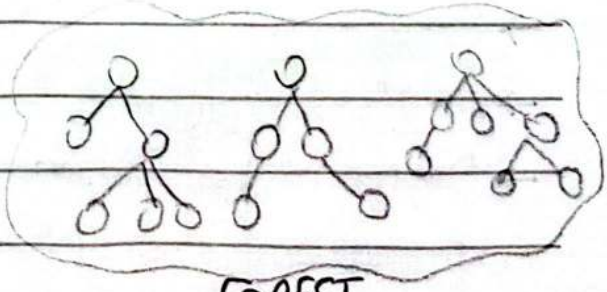


height of a Node

Distance from leaf node

A: 2 B: 1 E: 0

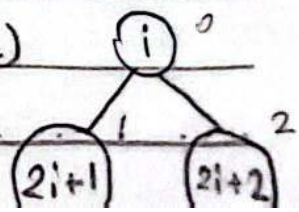
```
typedef struct Node {
    int value;
    Node *left;   child
    Node *Right;  sibling
} Node;
```



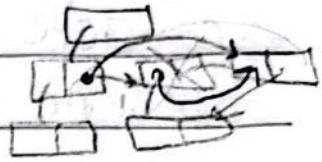
FOREST

✱ Binary tree → en çok kullanılır (max child = 2)

- daha az alabilir -

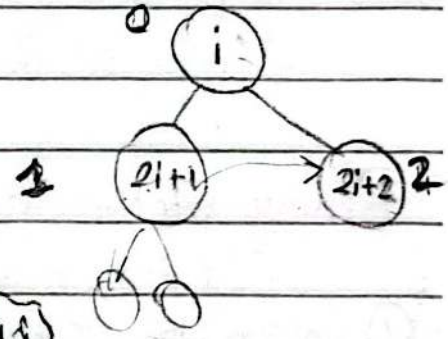


(*) ağaçta aradan Parent silme veya ekleme problemi : Linked List bağlar kopar yeni bağlar bağla...



// Binary tree with Linked List

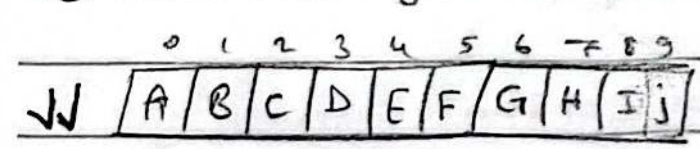
```
typedef Struct BTNode {
    int value;
    BTNode *left;
    BTNode *Right;
} BTNode;
```

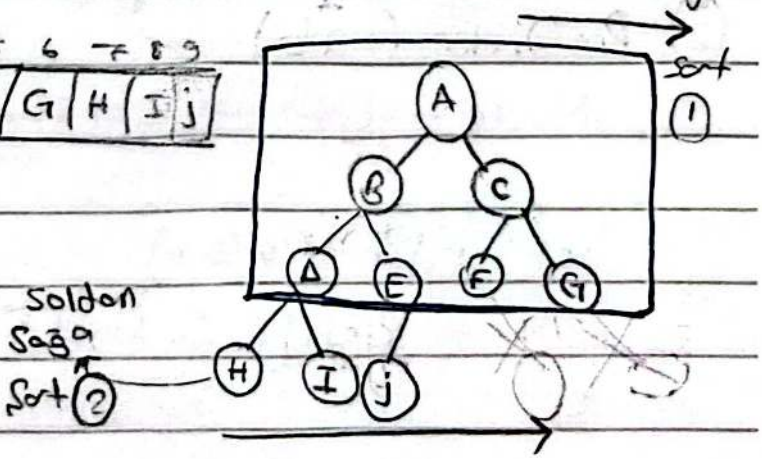


(*) Complete Binary Tree : (ÖNEMLİ)

① All levels from 0 to MAX -1 are completed

② Last level gets completed from left to right





→ max nodes = $2^{h+1} - 1$

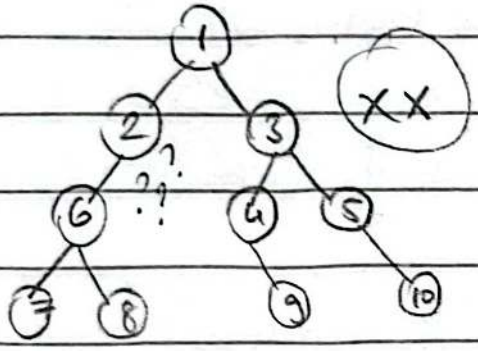
→ max leaves = 2^h

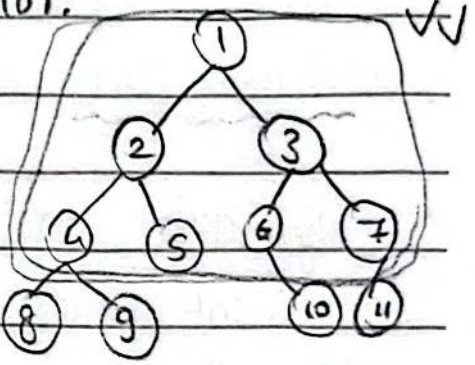
→ min leaves = 1 (root)

→ min. leaves (depend on h) = h+1

Balanced Binary Tree ÖNEMLİ

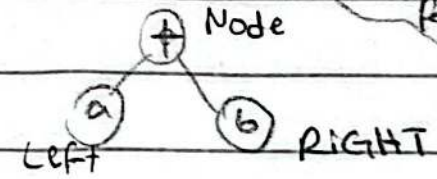
→ complete BT nin ilk şartının olması gerek ama ikinci şart olmasa da olur.





Sağ ve sol dalların yükseklik farkı ≤ 1

Binary Tree Traversal

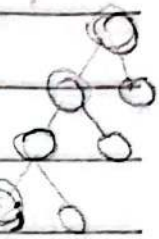


① inorder: (a+b) :
Left → Node → Right

② PreOrder (+ab)
Node → Left → Right

③ Post Order (ab+)
Left → Right → Node

```
→ void PostOrderPrint (Node *node) {
    if (node != NULL) {
        PostOrderPrint (node → left);
        PostOrderPrint (node → right);
        printf ("%d ", nod → value);
    } Return;
}
```



→ en sola gitmeyi

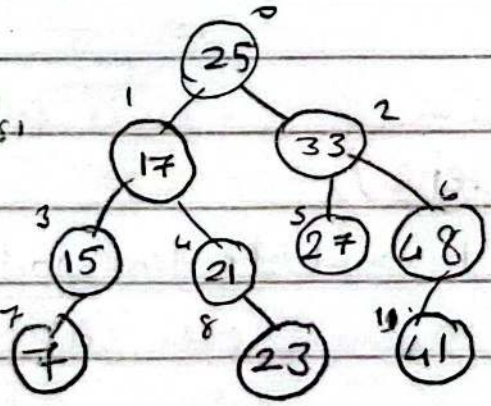
```
→ void PreOrderPrint (Node *node) {
    if (node != NULL) {
        printf ("%d ", node → value);
        preOrderPrint (node → left);
        preOrderPrint (node → right);
    }
}
```

recursion

Binary Search Tree

wost case : O(N) ← node sayısı

Balanced case : $O(\log_2 N)$

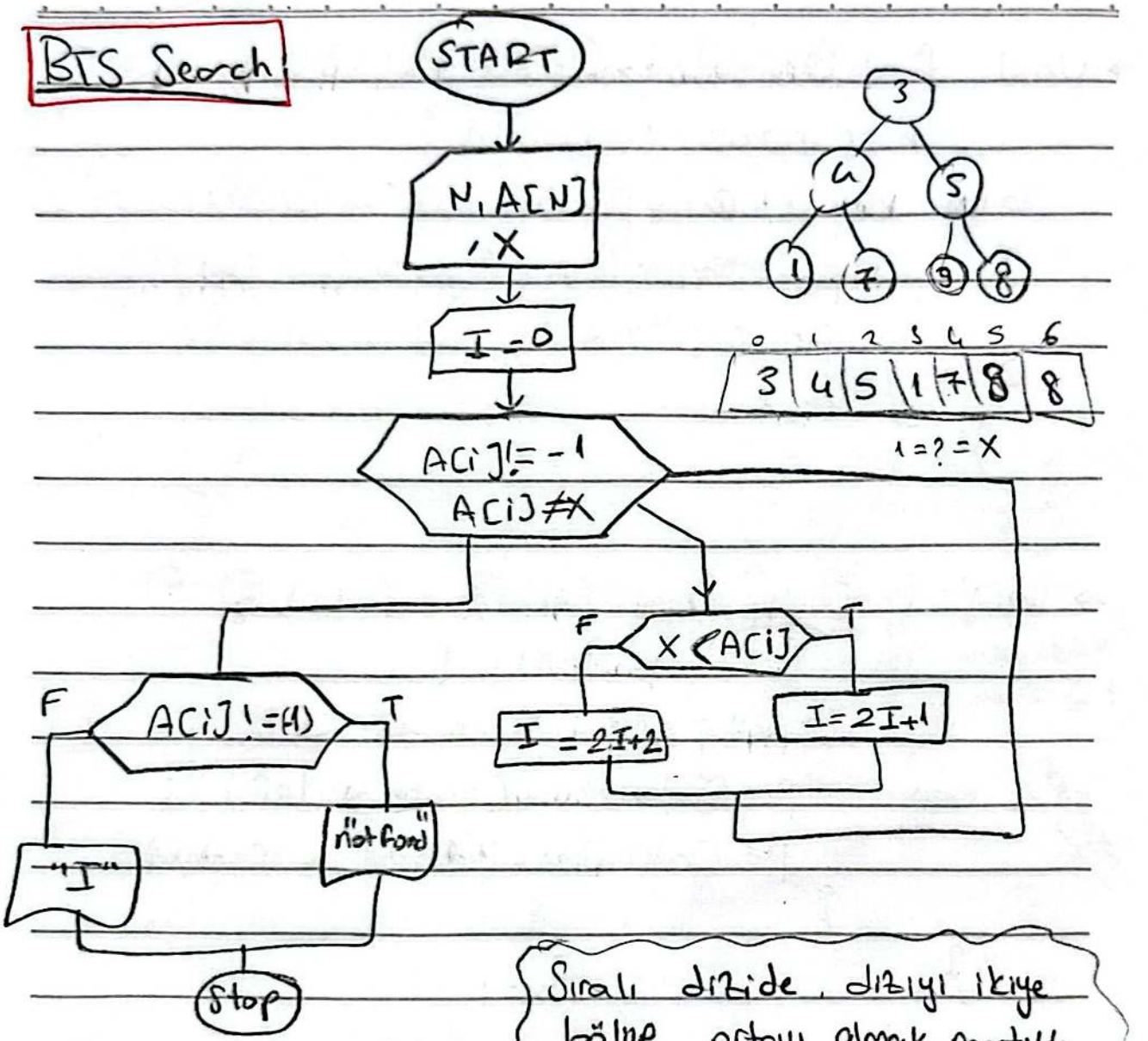


Sol → daha küçük Sağ → daha büyük

→ BST insert etmeyi

Scanned with CamScanner

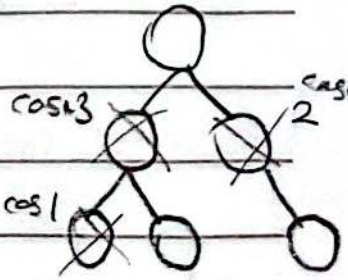
BTS Search



Sıralı dizide, diziyi ikiye bölüp ortayı almak mantıklı
Sol orta, sağ orta Sonra böl

Delete

- case 1 → delete leaf node
- Case 2 → delete interal node (1 child)
- Case 3 → delete interal node (2 child)

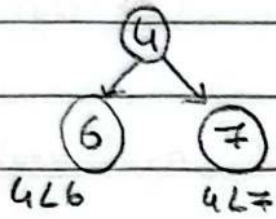


leaf son. node. Siler

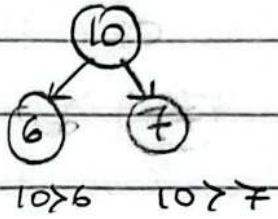
```
struct node *deleteNode(struct node *root, int key) {
    if(root == NULL) return root;
    if( key < root -> key )
        root -> left = deleteNode(root->left, key);
    if (key > root -> key)
        root -> right = deleteNode(root -> right, key);
    else{
        if( root -> left == NULL)
            struct node * tmp = root -> right;
            free (root);
            return tmp;
        else if (root -> right == NULL)
            struct node *tmp = root -> left;
            free(root);
            return tmp;
        Struct node * tmp = minValue Node(root->right);
        root -> key = tmp -> key;
        root -> right = deleteNode( root->right, tmp -> key);
    } return root;
}
Struct node *minValue Node( struct node * node) {
    Struct node * current = node;
    while (current && current -> left != NULL) {
        current = current -> left;
        return current; }
}
```

heap tree Complete Binary Tree olmalı

min heap tree



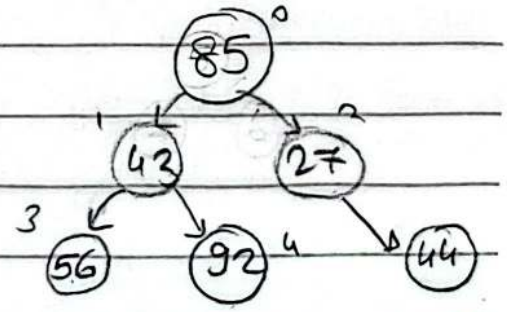
max heap tree



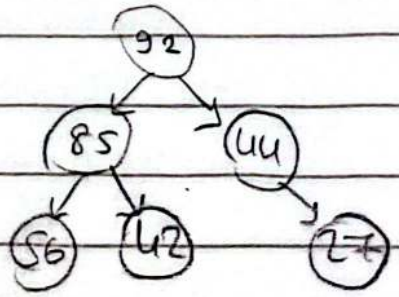
- heapes are efficient in maintaining the highest or lowest priority element athe tree
- it is also a complete Binary tree (Balanced + left)
- finds kth largest in an array
- merges and implements priority queue

→ heap tree oluşturma:

- N/2 → 4/2 = 2
- 27 < 44 replace
- 42 < 92 replace
- 92 > 85 replace



Bu işleme shift-up denir



O(n log m) heap tree search işleminde kullanılır

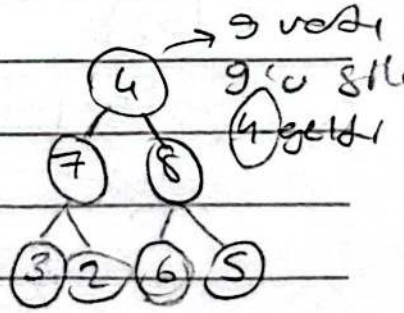
```
Void insert( int item, int Heap[], int *n){
        Heap[*n] = item; //en sona ekliyorut
        shiftUp(item, Heap, *n); // agaci yeniden sirala
        (*n)++; //size ++
}

Void ShiftUp (int item, int Heap[], int *n, int adr)
        int stop = 0;
        while((adr>0 && (!stop))
            parent = (adr-1)/2
            if(Heap[parent] < item)
                swap(&Heap[adr], &Heap[parent]);
                adr = parent;
            else
                stop = 1;
```

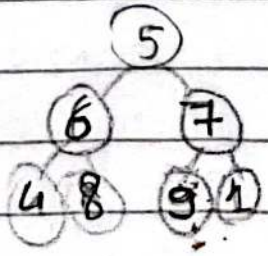
Delete

```
void max_heapify (int dizi[], int i, int N) {
  int left = 2 * i;
  int right = 2*i +1
  if( left <= N && dizi[left] > dizi[i]){
        largest=left; }
  else {
        largest = i ; }
  if( right <= N && dizi[right] > dizi[largest]) {
        largest = right;
  if( largest != i ) {
      swap ( dizi[i] , dizi [largest]);
      max_heapify ( dizi, largest, N);
}
}
}
```

log(n) → heapify

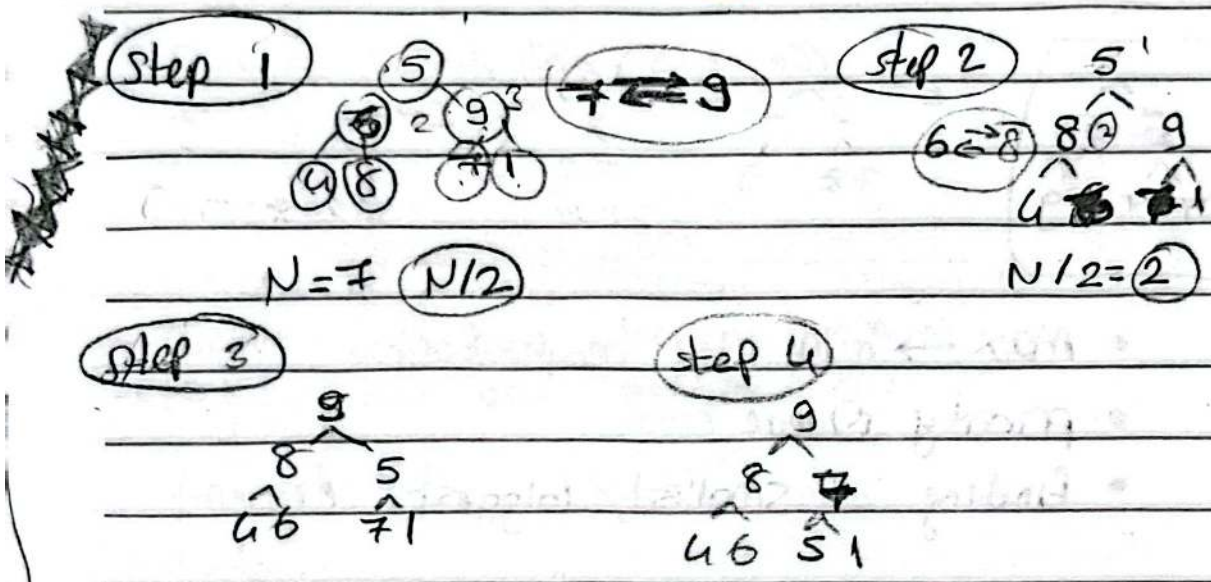


Peki yapısı tamamen karışık ağacı tekrar sıralamak istersek...



heap Sort

```
void build_maxHeap(int diz[]) {
    for(i=N/2; i>=1; i--)
        max_heapify(diz, i, N);
}
```



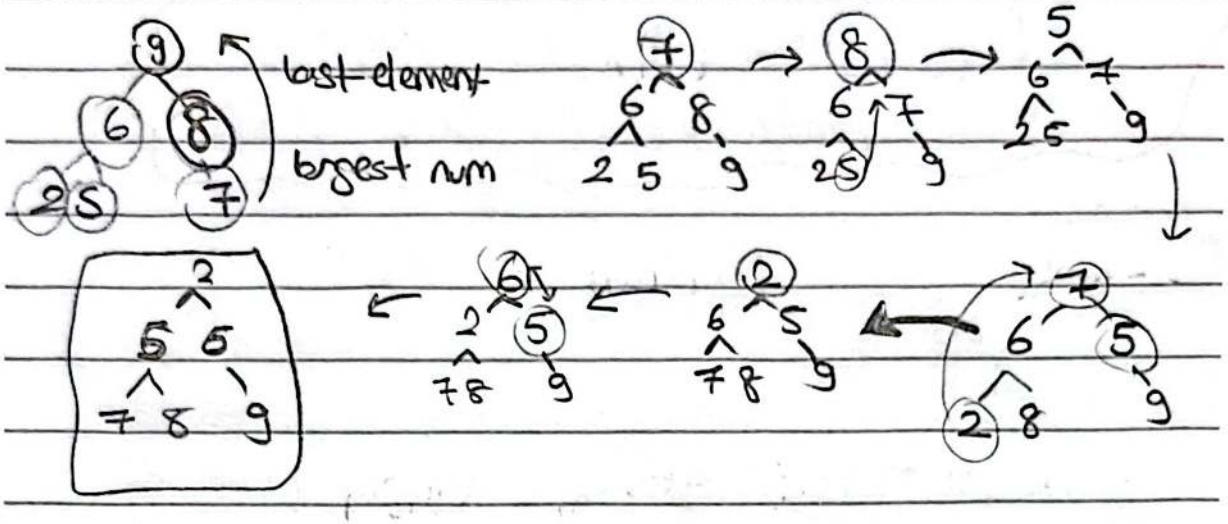
- **Advantages**
  - → fast access to min/max
  - → efficient insertion / deletion
  - → real time app.
- **Disadvantages**
  - → not suitable for searching
  - → extra memory
  - → slower

Applications of heap Data structures:

- Priority Queues
- Sorting Algorithms
- Graph Algorithms
- File compression
- Load Leadning
- Stock Market

Scanned with CamScanner

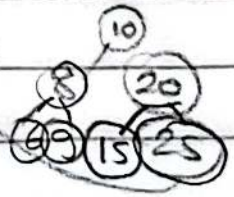
Heap Sort [O(n log n)]



- max → min lay heap sort
- priority Queue
- finding k smallest/biggest element

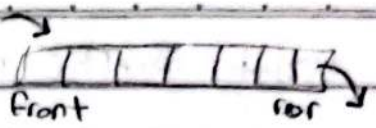
1- check if a given Binary Tree is a Heap (complete)
2- find kth smallest/longest element
3- convert Min Heap to Max heap ~

pop → O(1) çünkü en sağdaki eleman



↳ dağından en yüksek değere erişmek için

Priority Queue



- heap data structure
- Linked List
- Array
- Binary search tree (AVL ...)

Operations:

	array	linked	BST	binary heap
→ insert:	O(1)	O(n)	O(logn)	O(logn)
→ remove:	O(n)	O(1)	O(logn)	O(logn)
→ Peek: (elişmek için)	O(n)	O(1)	~~O(logn)~~ O(1)	O(1)

Aplication:

process Nargement and interrupt handling

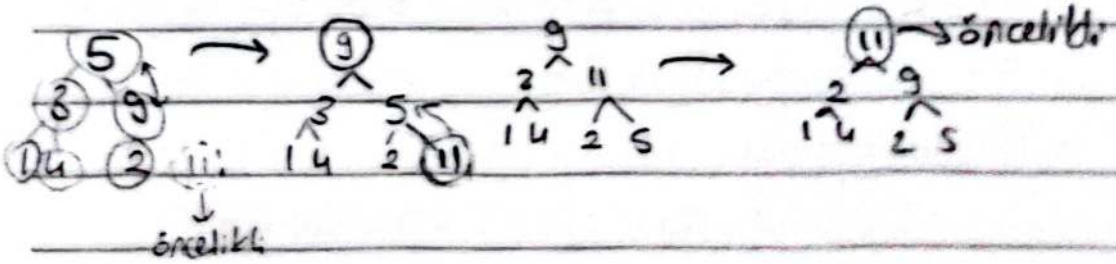
Dijkstra (shortes path)

huffman coding (data compession)

banking operation

Airplane Queues (Business, ecoromy)

Gelen kişinin yeniden öncelikli olarak konumlandırılması.

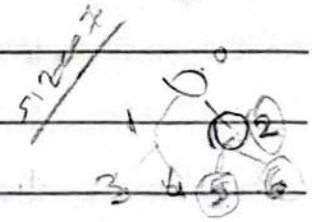


Peek → en yüksek öncelikle değere ulaşma

```
Void insert(int array[], int newNum){
  if (size == 0)
    array[0] = newNum;
    size = +1;
  else
    array[size] = newNum;
    size = +1;
    for(i = size/2 - 1; i >= 0; i--){
      heapify(array, size, i);
    }
}
```

bana gerek yok
çünkü bu sınıfı bir dizi diğer internal'lar gezmeye gerek yok

```
Void heapify(int array[], int size, int i){
  if (size == 1)
    Print("single element")
  else
    largest = i;
    L = 2i+1; R = 2i+2;
  if (L < size && array[L] > array[largest])
    largest = L;
  if (R < size && array[R] > array[largest])
    largest = R;
  if (largest != i)
    Swap(&array[i], &array[largest])
    heapify(array, size, largest);
}
}
```



```
void deleteRoot(int array[], int size){
    swap(&array[0], &array[size-1]); // son node ilk node ile degisti
    size = -1;
    for (i = size/2 - 1; i >= 0; i--){
        heapify(array, size, i);
    }
}
```

check if a given binary tree is a Heap ?

- struct Node
- new node()
- countNodes()

```
unsigned int countNodes(struct Node* root)
    if (root == NULL)
        return(0);
    return(1 + countNodes(root->left) +
        countNodes(root->right));
```

```
bool isCompleteUtil(struct Node* root, unsigned int index,
unsigned int number_nodes)
    if (root == NULL) return(true);
    if (index >= number_nodes) return (false);
    return(isCompleteUtil(root->left, 2*index+1, numb...)
        && isCompleteUtil(root->right, 2*index+2, numb...));
}
```

```
bool isHeap( struct Node* root) {
    if (root->left == NULL && root->right == NULL)
        return (true);
    if (root->right == NULL) {
    return (root->key >= root->left->key); }
    else
        if(root->key >= root->left->key
        && root->key >= root->right->key)
        return ( isHeap(root->left && isHeap(root->right));
        else
            return (false);

bool isHeapLiterally( struct Node* root)
    unsigned int node_count = countNodes(root);
    unsi "    " index = 0;
    if (isCompleteUtil (root, index, node_count)
                && inHeap(root)) return(true);
    return (false);
}

    int main()
{   struct Node root = NULL;
    root = Newnode(9);
    root->left = newnode(8);
    :
    :
    if(isHeapLiterally(root)
    else
```

control

hafta -11

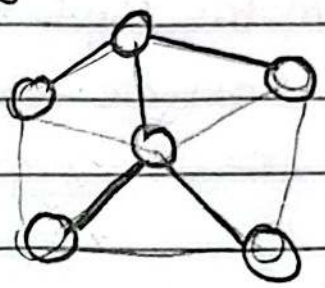
Graflar

Minimum Spanning Tree : spanning tree with the min sum of weights

→ Acyclic graph

→ includes all vertices

• en kısa yol → tüm düğümlere erişmeli

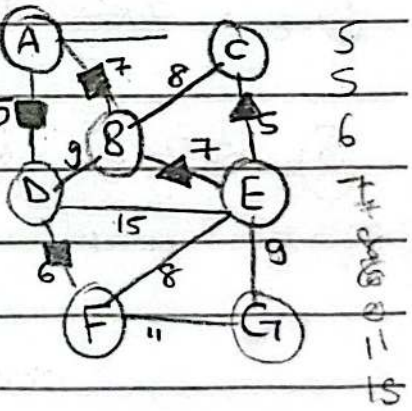


Örnek :

① Kruskal's Algorithm : Greedy Algorithm $O(E \cdot \log V)$

A locally optimal choice is globally OPTIMAL

• cycle oluşturmayacak şekilde düğümlere ulaş



başta hepsi sıfır

A	B	C	D	E	F	G
1	0	0	1	0	0	0

A-D = 5 ✓

A	B	C	D	E	F	G
1	0	2	1	2	0	0

C-E = 5 ✓

A	B	C	D	E	F	G
1	0	2	1	2	1	0

D-F = 6 ✓

A	B	C	D	E	F	G
1	1	2	1	2	1	0

A-B = 7 ✓

A	B	C	D	E	F	G
1	1	(1)	1	(1)	1	0

B-E = 7 ✓ * etiket farklı, birbirine eşitle

A	B	C	D	E	F	G
1	1	1	1	1	1	1

E-G = 9 ✓

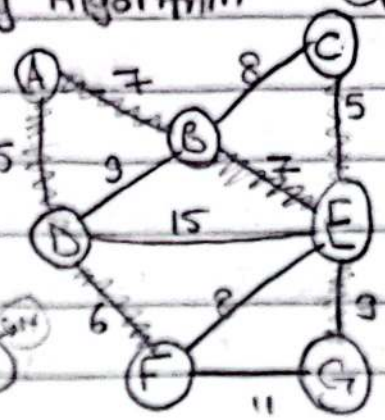
39

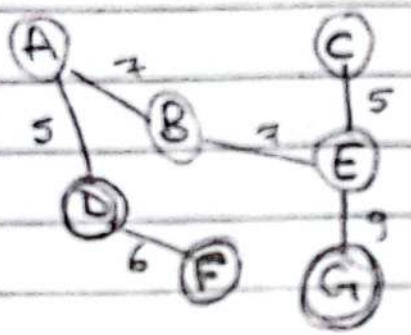
A locally optimal choice is globally optimal

② Prim - Algorithm : Greedy Algorithm $O(V^2)$

- start from one Node
- Grows MST one edge at atime until all nodes are included

Always choose the edge with the (min amount possible)





39

1. start with A
A - D = 5 ✓
D - F = 6 ✓
A - B = 7 ✓
B - E = 7 ✓
E - C = 5 ✓
G - F = 9 ✓

→ Bunun için komşu linkli listeler kullanılır çünkü burdaki amaç değerleri birbirine bağlamak değil sadece tek bağlantı yaparak tüm değerlere ulaşmak

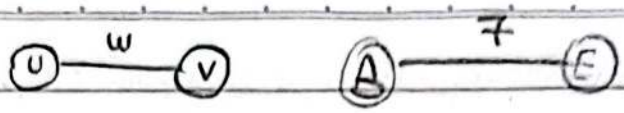
A → [B | 7] → [D | 5] → ...

→ başlangıç noktası rastgele olduğu için bazen en kısa yolu bulamaz (cycle oluşturur ama kısa kalır)

→ N vertex ⇒ N-1 edges 7 değer - 6 kenar

Dense graph → Adj Matrix (more edges)
Sparse Graph → Adjency List (Few edges)

Scanned with CamScanner

Kruskal : 

Javam

case 1 : label[u] = label[v] = 0 // boşta

case 2 : label[u] OR label[v] = 0 // boşta

(*) case 3 : label[u] ≠ label[v] → 2 → 1

case 4 : label[u] = label[v] // cycle olur

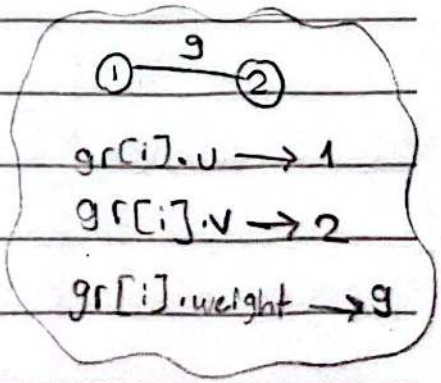
↳ ~~label[u] ≠ label[v] != 0~~ gerek yok

MST has exactly (n-1) edges

Arrange E in the order of increasingly costs

```
for(i=1; i<=n; i++) {
    select the next smallest edge
    if the edge connects (two different)
    connected components
        add the edge to (MST)
}
```

Kruskal Algorithm



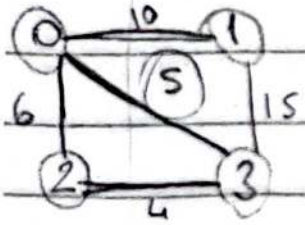

```
typedef struct Edge{
    int u, v;
    int weight;
} Edge;
```

label = etiket

```
Void kruskal(Edge gr[], mst[], int nV){
  int labelNo = 1; int i, j;

  for(i=0 i<NV; i++)
      label[i] = 0;

  i=j=0;
  while(i< NV)
      uu = gr[j].u;
      vv = gr[j].v;
      if(label[uu] + lobd[vv] == 0)
          mst[i].u = uu;
          mst[i].v = vv;
          mst[i].weight = gr[j].weight;
          label[uu] = label[vv] = labelNo++;
          i++;
      ; if(lobd[uu] != label[vv]){
          mst[i].u = uu;
          mst[i].v = vv;
          mst[i].weight = gr[j].weight;
          i++;
          if(label[uu])
              label[uu] = label[vv];
          else if(lobd[vv])
              label[vv] = label[uu];
          else
              union(label, nV, uu, vv);
      j++
```



g[1].weight=4
g[1].v=3
g[1].u=2

1	2-3	4	i=1 i=2
2	0-3	5	i=3
3	0-2	6	i=4
4	0 1	10	i=5
5	1 3	15	

hocaya locivert kalem al.

·union(

MST Algorithms Running Time

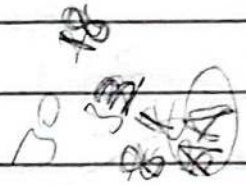
Prim's Algorithm

V-1 iterasyon → dist O(V) . O(V)

$O(V^2)$

Kruskal Algorithm

$O(E \log V)$ time (sort)



Graph Traversal

1- Depth First Search : on a stack

2- Breadth First Search : on a Queue

BFS is better when target is closer to Source.

DFS is better when target is far from Source.

effective algorithm

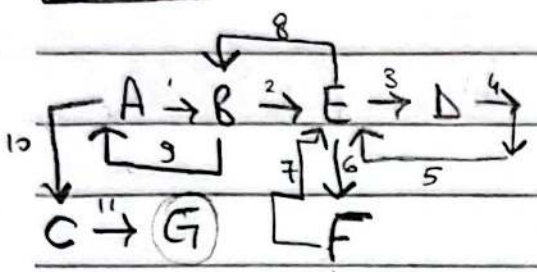
* tremaux maze exploration !
* sixth degree of freedom

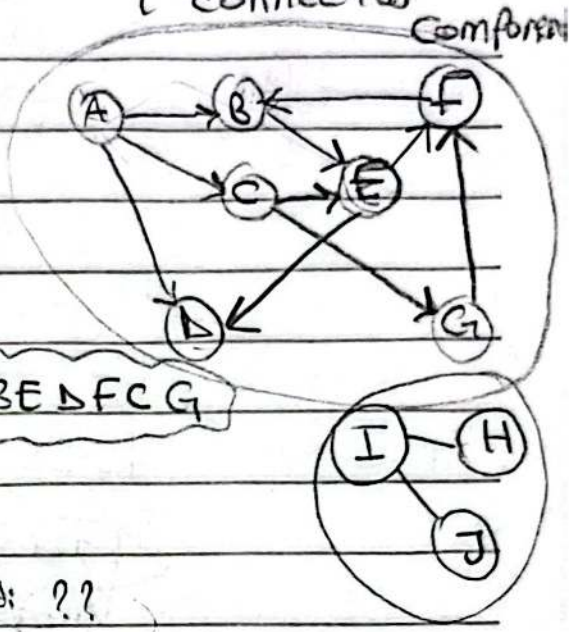
heap tree

Run length encoding / silver

⊛ Sınavlarda (DFS) sorular hoca bak sayer

DFS O(E)





1 connected component

A B E D F E B A C G → A B E D F C G

① recursive

DFS(G, S) — graph, source

girdi ??

{

→ Mark S as VISITED

→ for all neighbours w of S in Graph G

if w is NOT visited ↓

DFS(G, w); print("w")

}

visited

A	B	C	D	E	F	G	H	I	J
1	1	1	1	1	1	1			

1. component

1. comp.

② non-recursive DFS

DFS(G, S) {

stack.push(s);

mark S as visited

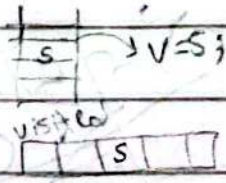
while(stack is not empty){

v = stack.pop(); for all neighbours w of v in G

if (w is not visited)

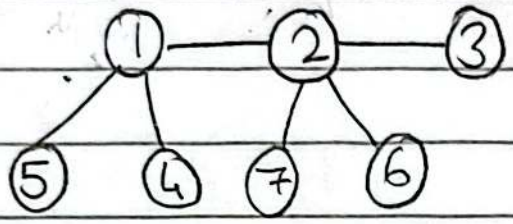
stack.push(w);

mark w as visited



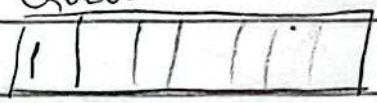
Scanned with CamScanner

BFS : Like level order in binary tree

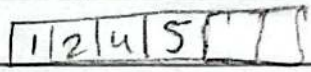


Queue

→ Start with (1)



the neigbors : 2, 4, 5



→ go to all neigbors (1), (2), (4), (5)

→ continue with (2)

the neighbors : 3, 7, 6



→ go to all neighbors 1 (2), 4, 5, 3, 7, 6

Seviye seviye gider ama Dfs en derine iner

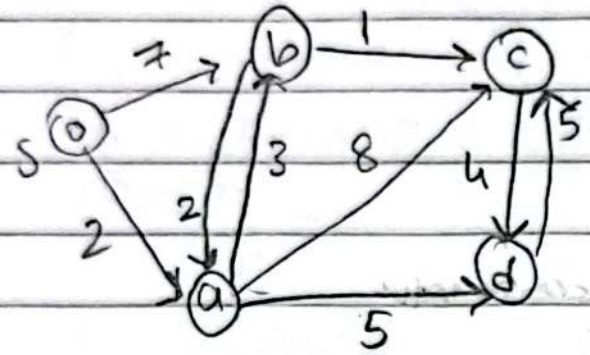
Dfs : 1, 2, 3, 6, 7, 4, 5

(*) the rule of BFS is : when u visit a node you must visit all of its adjcence nodes.

applications :

- testing a graph for bipartiteness
- copying garbage collection

Son hafta



→ Dijkstra → pozitif ağırlıklar için çalışır

→ başta hepsine sonsuz ata

adjacency list

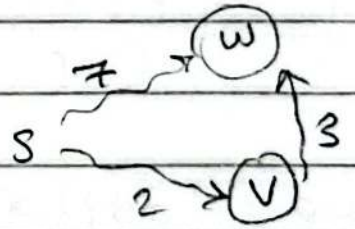
s → a:2 → b:7

a → b:3 → c:8 → d:5

b → a:2 → c:1

c → d:4

d → c:5



Step:0 initialization

v	s	a	b	c	d	
d[v]	0	∞	∞	∞	∞	
pred[v]	NULL	NULL	NULL	NULL	NULL	
color[v]	w	w	w	w	w	white

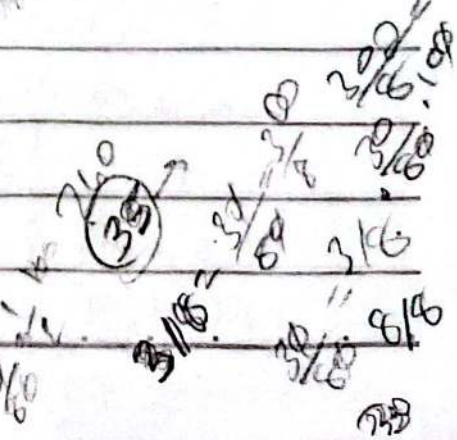
if dist[sw] > dist[sv] + dist[vw]

dist[sw] = ↓

pred[w] = v;

Priority Queue

v	s	a	b	c	d
d[v]	0	∞	∞	∞	∞



Step 1: adj[s] = {a, b};

V	s	a	b	c	d
d[v]	0	2	7	∞	∞
	0	2	5	(10) → 6	7
pred	N	s	s	N	N
	N	s	a	a	a
		s	a	b	a
color	B	w	w	w	w
	B	B	w	w	w
	B	B	B	B	B

decrease value

queue	a	b	c	d
	2	7	∞	∞

queue	b	c	d
	5	7	10

queue	c	d
	b	7
	10 > 7	

queue (7)

Step 2 : adj[a] = {b, c, d}
Step 3 : adj[b] = {a, c} →
Step 4 : adj[c] = {d}

Q = {with all vertices}

```
Dijkstra (G, w, s)
 { for (each u ∈ V)
    { d[u] = ∞
      color[u] = WHITE;
    }
  d[s]=0;
  pred[s]=NULL;
```

Q = (QUEUE, with all vertices);

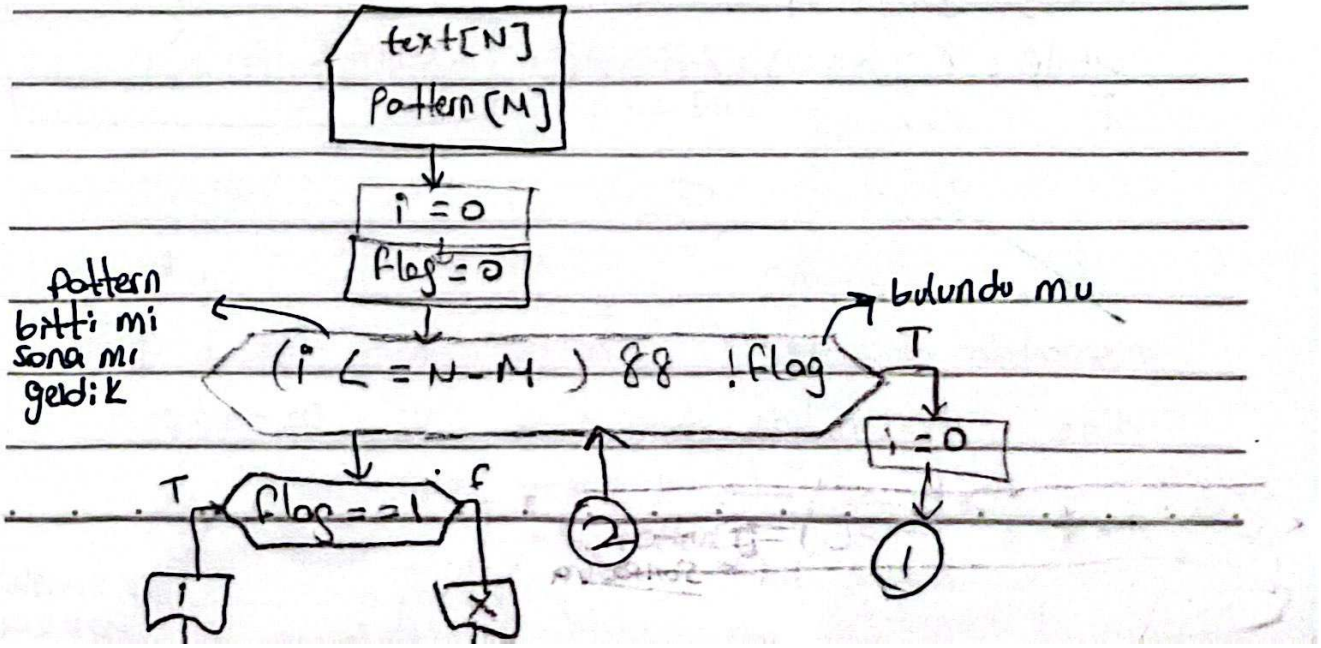
kod.

```
while(non-empty(Q))
{ u = ExtractMin(Q) → minimum değeri Q'dan
    forleach v ∈ adje(u])        çıkar
    {
      if
```

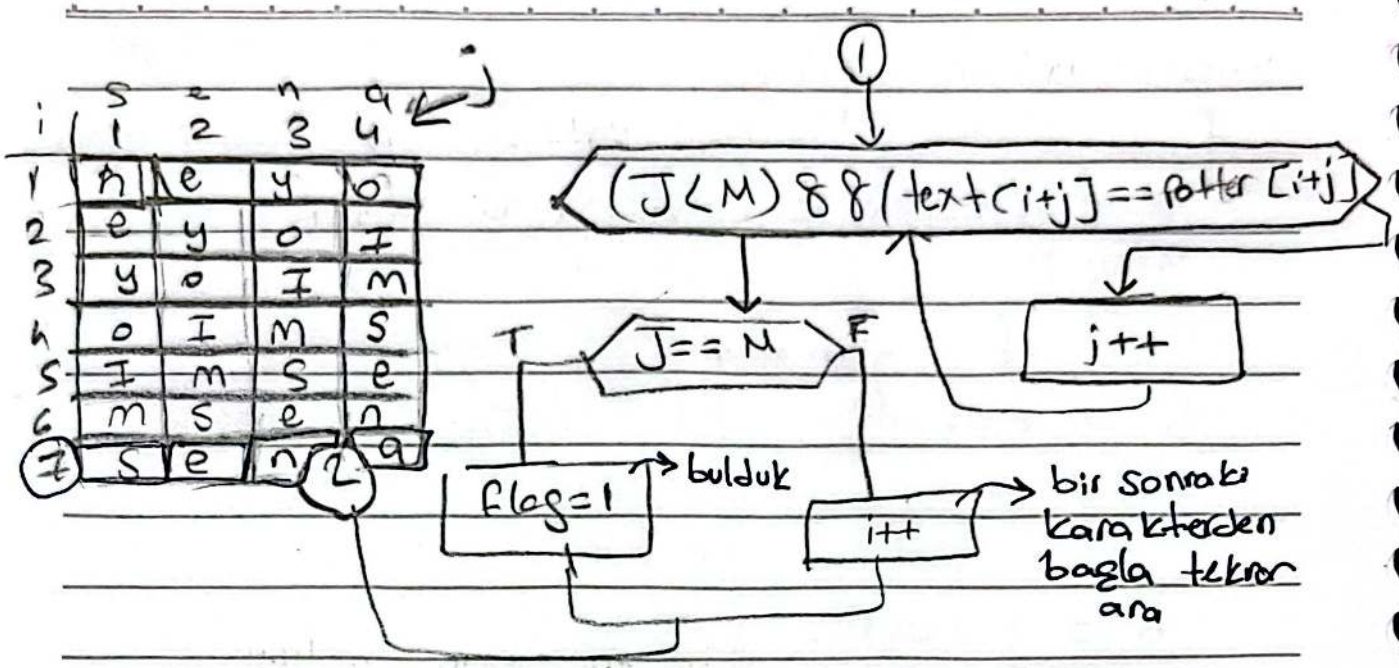
Substring Search

① Brute-Force Search

typical: N (text we search)
worst case(N*M) M = the sentence



Gok sorular



text NAHANEEDLE

string NEENE

NEEDLE

NEENE

NEEDLE

NEEDLE

buldu √√

heyoImsena = 10

sena = 4

10 - 4 + 1 = 7

örnekleyelim

2 Boyer - moore Algorithm (M = size of pattern) Needle

i = 0; j = M-1;

→ son hafi karşılaştır

while ((j >= 0) && (p[j] == t[i+j]))

{

...

j--;

...

}

mismatch character !

case 1: mismatch char not in the PATTERN

. . . text → i → i + (j + 1) . . . . . .

uyuşmayan harfin sonrasına kelime boyu kadar 'o' needle'da yoksa

Case 2-a

mismatch char in the pattern

$i = i + (j - \text{rightmostOccurence in the pattern})$

i=6

heyo N LE 'N' needle'da var

NEE D LE

$i = 6 + (3 - 0) = 9$

NEEDLE

↓ 'N' = 0

9 'j' = 3

Yani kaydırıyoruz ama uyuşmayan karakter ile pattern'in ilk karakteri çakışacak yere kaydır.

Case 2-b

mismatch char is in the pattern but the jump = negative

$i = i + 1$

heyo L $2 - 4 = -2$

NEEDLE

2 4

Hoorpool Algorithm

NEEDLE
0 1 2 3 4 5

	A	B	...	D	E	...	N
Jump	-1	-1		3	5		0

(dizi)

Skip table computation:

28 ← only uppercase ← size of lookup table

```
Void shiftTable(char P[], int right[], int offset, int size)
    int i; M = strlen P;
    for(i=0; i<size; i++)
        right[i] = -1;
    for(i=0; i<M; i++)          → for this case offset = A
        right[P[i] - offset] = i      'A' = 0
                'N' - 'A'
```

Bad Symbol shift → ters eşleşme bilgisi kullanılır.
Good Suffix shift → örüntü içindeki benzerliği kullanır

BAOBAB = 6 k = eşleşen değer sayısı

bad symbol shift

c	A	B	C	D	...	O	...	Z
$t_1(c)$	1	2	6	6	6	3	6	6

$d_1 = \max\{t_1(c) - k, 1\}$

Good Suffix table

k=1 BAOBA$\bar{B}$ $d_2 = 2$ kaydırma

k=2 BAOB$\bar{A}\bar{B}$ $d_2 = 5$ çünkü başta B var

k=3,4,5 BAOBAB $d_2 = 5$ " " "